

Does Separating Planning from Execution Help LLM Agents Solve Multi-App Tasks?

An ablation study on LARA, a multi-agent system for the AppWorld benchmark

Abstract

A natural starting point for an LLM agent that must discover APIs, form a plan, and execute it is a single agent that does all three in one continuous reasoning loop — one system prompt, one context, one model call pattern, minimal engineering overhead. We ask whether this simplicity costs accuracy, by comparing such a unified agent against LARA's actual design: a dedicated **Explorer** stage that discovers relevant APIs and writes an explicit, app-tagged plan, handed off to a separate **Executor** stage that carries out that plan via a ReAct loop. To isolate the *architectural* question from a *knowledge* question, the unified agent is given the Explorer's full planning and discovery prompt verbatim, plus nearly double the execution budget to compensate for having to interleave discovery with execution. On a fixed, paired, 45-task slice of AppWorld run same-day for both conditions, the two-stage design scores **73.3%** task completion against the unified agent's **42.2%** — a 31.1-point gap, significant under an exact McNemar test ($p = 0.0043$). We conclude that a dedicated planning stage is not mere engineering overhead: it materially improves an LLM agent's ability to complete long, multi-application tasks, and the mechanism appears to be that unconstrained interleaving of discovery and execution lets errors compound in ways an explicit upfront plan prevents.

1. Motivation

Consider the simplest reasonable design for an agent that must complete a task like "pay everyone in *debts.csv* via Venmo, or file a Splitwise expense if they lack a Venmo account": a single LLM in a loop, given access to a code-execution environment and instructions on how to look up API documentation on demand. At each step it writes a small piece of code, observes the result, and decides what to do next — a standard ReAct-style agent. This has real virtues: it is the simplest thing that could work, it requires no orchestration between multiple LLM roles, and it lets the model interleave "figure out what API to call" with "call it" as opportunities arise, rather than committing to a plan before it has any evidence from the environment.

LARA does not use this design. It splits the work into two stages that never share a reasoning context: an **Explorer** that only reads API documentation and writes a plan — it executes no code — and an **Executor** that receives that plan and carries it out, dispatching each step to a specialist prompt for the relevant app. This adds real cost: a second model role to maintain, a hand-off point where information can be lost, and less flexibility for the agent to react to what it discovers mid-execution by revising the plan on the fly.

The question this paper answers is whether that cost is justified. If a single agent given the same knowledge can match the two-stage design's accuracy, the split is unnecessary complexity. If it cannot, the split is solving something a single reasoning loop does not solve on its own — and this paper is about identifying what that something is.

2. Related design space

Three broad families of agent design bear on this question. **Monolithic ReAct agents** (Yao et al., 2022) interleave reasoning and action in a single loop with no separate planning phase — the design our single-agent condition approximates. **Plan-then-execute agents** separate a planning call from execution, but the plan is typically a static list handed off once, without the specific per-step app tagging and specialist dispatch LARA uses to route execution knowledge. **Multi-agent frameworks** more broadly (e.g., supervisor/worker patterns) split responsibility

across roles for reasons of context management or specialization, but rarely isolate *planning itself* as the sole variable — role splits are usually confounded with differences in the knowledge or tools each role receives. This study's contribution is methodological: it holds knowledge, tools, and specialist prompts fixed across conditions, varying only whether planning happens as a separate stage before execution begins.

3. Experimental design

3.1 The confound this study is built to avoid

The central risk in any single-agent-vs-two-agent comparison is that the two conditions differ in more than architecture — the single agent is often also given *less information*, because the planning stage's prompt is not transplanted into it. A single agent that loses to a two-stage system under those conditions has not shown that planning-as-a-separate-stage matters; it has shown that having less information matters, which is a different and less interesting claim.

This study avoids that confound directly: the single-agent condition's system prompt is built by calling `build_explorer_system()` — the exact function that generates the Explorer's own prompt — and concatenating its output with the Executor's prompt. The single agent therefore has access to *every piece of planning and discovery guidance* the two-stage design has, expressed in identical language. The only thing it lacks is a separate LLM call in which to apply that knowledge before execution begins; planning and execution instead occur, if at all, interleaved within a single continuous reasoning trace.

3.2 Compensating for the step-budget asymmetry

In the two-stage design, the Explorer's API discovery happens entirely within its own LLM calls, which are not counted against the Executor's step limit. The Executor's 16 allotted ReAct steps are therefore 16 steps of pure execution. The single agent has no such luxury: any API discovery it performs necessarily consumes steps from the same budget it uses for execution. An initial equal-budget pilot showed several tasks exhausting all 16 steps without producing an answer — a budget-exhaustion failure, not evidence bearing on the planning question. The single agent's step budget was therefore raised to 30, giving it roughly the same amount of *execution* budget the two-stage design receives for free. This is a deliberate handicap removed in the single agent's favor, not a thumb on the scale for our hypothesis.

3.3 Mitigating a residual routing confound

The single agent begins with a placeholder plan whose step ordering follows a fixed per-app keyword table rather than the task's actual logic, which can cause it to be routed to the wrong specialist prompt at the start of a task — an artifact of the ablation harness, unrelated to the planning-separation question itself. To reduce this, the harness re-parses the first plan the agent writes in its own output and rebuilds its internal specialist-routing map from that self-authored plan rather than the keyword-ordered placeholder. This mitigation engaged on 28 of the 45 tasks in this run; on the remaining tasks the agent did not produce a re-parseable plan of its own, and some residual routing-order penalty may remain on those tasks (see Limitations).

3.4 Slice, model, and controls

Both conditions ran the identical fixed 45-task slice of AppWorld's training split (never the held-out test splits used for the project's leaderboard submission), in identical task order, using gpt-4.1-mini for all model calls, on the same day. Held constant across both conditions: one execution attempt per task (no retry), no reviewer/critique stage, all ten app-specialist prompts, the cross-application memory ledger, and the code-execution bootstrap helpers. The only architectural variable is whether planning occurs as a dedicated stage before execution or is left to emerge, if it does, within a single continuous reasoning loop.

Token usage was recorded from the provider's own per-call usage figures for every request in both conditions, giving a cost comparison independent of the accuracy comparison.

4. Results

Condition	Task completion	Scenario completion	Tokens / task
Two-stage (Explorer + Executor)	73.3% (33/45)	64.0%	72,490
Single agent, same knowledge	42.2% (19/45)	40.0%	76,560

The two conditions were compared on the identical set of tasks, so an exact McNemar test on the paired outcomes is the appropriate significance test: 18 tasks were solved by the two-stage design and not the single agent, 4 were solved by the single agent and not the two-stage design, 15 were solved by both, and 8 by neither. This yields $p = \mathbf{0.0043}$, comfortably below conventional significance thresholds and supported by a substantial number of discordant outcomes (22) rather than a marginal handful.

Per-difficulty scores show the gap present at every level, largest at the easiest tier (92.3% vs 46.2% at difficulty 1) and narrowing somewhat at the hardest (56.2% vs 43.8% at difficulty 3) — a pattern discussed further below.

Notably, token cost was nearly identical between conditions (72,490 vs 76,560 per task, a 5.6% difference), despite the single agent's larger step allowance. The two-stage design is simultaneously the more accurate and the more economical configuration; the accuracy gap cannot be explained by one condition simply being given more computational budget to spend.

5. Discussion: why does planning as a separate stage help?

The result that a single agent with identical knowledge underperforms a two-stage design by 31 points rules out the simplest alternative explanation for LARA's architecture — that the two-stage split exists only to compartmentalize information the single agent could not otherwise hold. Since both conditions receive the same information, the gap must arise from *when and how* that information is used.

We propose the mechanism is **commitment before evidence accumulates versus commitment interleaved with it**. The Explorer produces a complete, ordered, app-tagged plan before any code executes — errors in that plan are visible and correctable as a static artifact, and the plan's structure (an ordered list of steps, each tagged to the app it concerns and its data dependencies) gives the Executor an external scaffold to check its own progress against. A single agent reasoning and acting in one continuous stream has no such external artifact: a wrong assumption made at step 3 is not a plan to revise, it is simply the context the model continues to reason from, and subsequent steps inherit its error silently. The unified agent is not lacking information; it is lacking a structural checkpoint at which a plan can be wrong in a way that is visible before any action has been taken on it.

This is consistent with the difficulty-level pattern in Section 4: the gap is largest on difficulty-1 tasks, where a two-stage design solves nearly every instance (92.3%) while a single agent still fails on more than half (46.2%). Difficulty-1 tasks are typically single- or few-step, single-app tasks where an error is not "hidden" by task length — the single agent's shortfall on tasks this simple suggests the effect is not primarily about running out of budget on long tasks, but about something more basic in how discovery-then-action differs from discovery-interleaved-with-action, even when the discovery in question is small.

5.1 Ruling out step-budget exhaustion directly

The difficulty-level argument above is indirect. A direct check is possible by examining the step number at which every failed task actually ended, against each condition's hard ceiling (16 for the two-stage design, 30 for the single agent, per Section 3.2). Across both conditions combined, **no task's failure occurred at its step ceiling**: the two-stage design's failures topped out at step 12 of its 16-step allowance, and the single agent's topped out at step 19 of its 30-step allowance. One task per condition ended without the agent ever calling `complete_task()`, but in both cases this occurred well before the step ceiling was reached, not because the loop was cut off mid-attempt.

In other words, in this run, **every observed failure was a wrong answer the agent was confident enough to submit, not an attempt that ran out of room to keep trying**. This confirms directly, rather than only inferring from the difficulty-level pattern in Section 5, that the step-budget compensation described in Section 3.2 succeeded in removing budget exhaustion as a confound: the 31.1-point gap reflects a difference in the *quality* of what each condition concluded, not in how many opportunities each condition had to reach a conclusion.

6. Limitations

The step-ordering mitigation is partial. The self-authored-plan rebuild mechanism (Section 3.3) engaged on 28 of 45 tasks; on the remainder the single agent may have continued operating under specialist routing derived from an arbitrary keyword-table ordering unrelated to the task's actual logic. This could inflate the measured gap on some fraction of tasks for reasons orthogonal to the planning-separation question. The size of the overall effect (31.1 points, $p = 0.0043$) makes it unlikely this fully explains the result, but the exact magnitude should be treated with this caveat in mind.

No within-condition repeat runs. A companion study on this same benchmark slice found that an unchanged two-stage configuration, run on three separate occasions with no code changes, scored 64.4%, 75.6%, and 71.1% — a spread of over ten points attributable to ordinary variance between runs of a hosted LLM, the mechanism of which (temporal drift versus simple stochastic variation between any two runs) our data do not distinguish. Both conditions in the present study were run in immediate succession specifically to minimize exposure to this variance rather than to eliminate it; no confidence interval from repeated sampling is available for either condition's reported score, and the *ratio* between conditions ($1.74\times$) should be treated as more approximate than the *qualitative* finding that a meaningful, significant gap exists.

Single model. All experiments use gpt-4.1-mini. Whether the planning-stage benefit persists, grows, or shrinks with a substantially more capable model is not tested here; a separate line of investigation in this project suggests that the contribution of *specialist domain knowledge* (a related but distinct architectural feature) shrinks — without disappearing — as model capability increases, which raises the same open question for planning separation specifically.

One asymmetry left uncontrolled. This study compensates the single agent with a larger step budget (Section 3.2) but does not test whether giving the two-stage design a *larger* budget than its default 16 steps would close any part of the gap. This is a distinct question from the one this study answers and is left to future work.

Train split only. All experiments use AppWorld's training split; the project's held-out test-split scores used for leaderboard submission were not touched by this or any other ablation in this line of work.

7. Conclusion

Holding knowledge, tools, and specialist prompts fixed, and correcting for the step-budget cost of doing API discovery inline, a single LLM agent solves 42.2% of a fixed AppWorld task slice, versus 73.3% for the same knowledge split across a dedicated planning stage and a separate execution stage — a 31.1-point gap that is statistically significant ($p = 0.0043$) and not explained by unequal token spend. The architectural decision to separate planning from execution is not compartmentalization for its own sake: it is responsible for a substantial share of the system's measured accuracy, and the evidence suggests the reason is that an explicit plan gives an agent something to be *visibly* wrong about before it acts, which a single continuous reasoning stream does not provide on its own.

Appendix: Reproducibility

- Task slice: `extended_slice.json` (45 tasks, difficulty-stratified, fixed seed)
- Model: `gpt-4.1-mini`, both conditions, same day (2026-09-06)
- Runner: `run_ablation.py {full,single} --extended`
- Analysis: `analysis/specialist_ablation_report.py` (exact McNemar, paired)
- Raw outputs: `experiments/outputs/ablation_{full,single}_ext/`
- Token logs: `tokens_planning{full,single}_ext.jsonl`